ClinicFlow

A clinic appointment booking platform, shipped end-to-end

DevOps / DevSecOps Final Capstone — Mayank Raj

Project Overview

What we built and why

- ClinicFlow is a clinic appointment booking system: book, check in, complete or cancel appointments across departments.
- Reception dashboard shows live KPIs — scheduled / checked in / completed — plus a per-entry status filter.
- Deliberately small application, real DevOps pipeline around it: the app is the vehicle, the pipeline is the lesson.
- One command (docker compose up --build) runs the whole stack locally; the same images are shipped to Kubernetes.

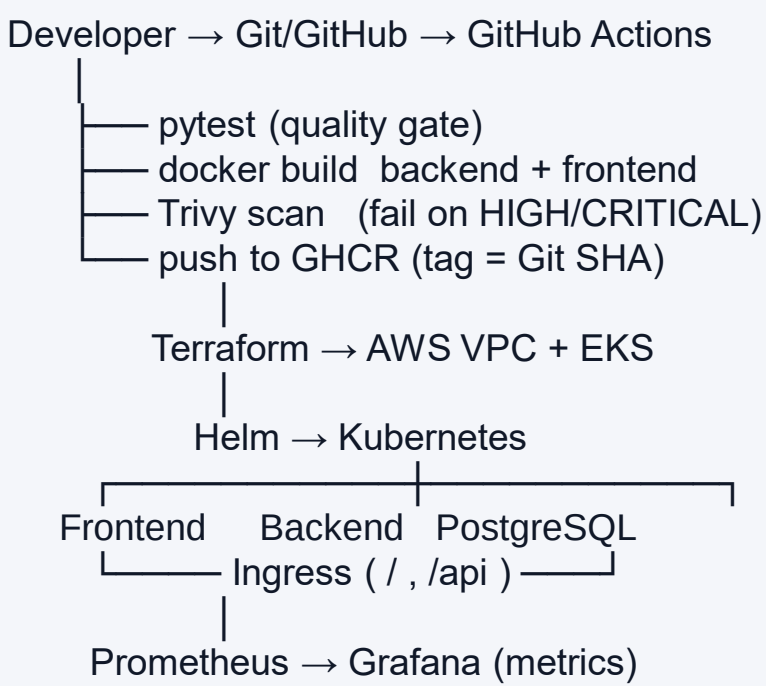
Tech Stack

Languages, frameworks and platforms

- Frontend — React 18 + Vite, hand-written responsive CSS, Nginx runtime.
- Backend — Python 3.12, FastAPI, SQLAlchemy 2 ORM, Pydantic v2 schemas.
- Database — PostgreSQL 16, schema managed by Alembic migrations.
- Tests — pytest + FastAPI TestClient against an isolated SQLite database.
- Containers — Docker multi-stage frontend, non-root images, Compose stack.
- CI/CD — GitHub Actions, Trivy image scanning, GitHub Container Registry.
- Infra & runtime — Terraform (AWS VPC + EKS), Helm, Kubernetes, Ingress, HPA.
- Observability — Prometheus (ServiceMonitor) + Grafana dashboards.

Architecture

From laptop to monitored cluster



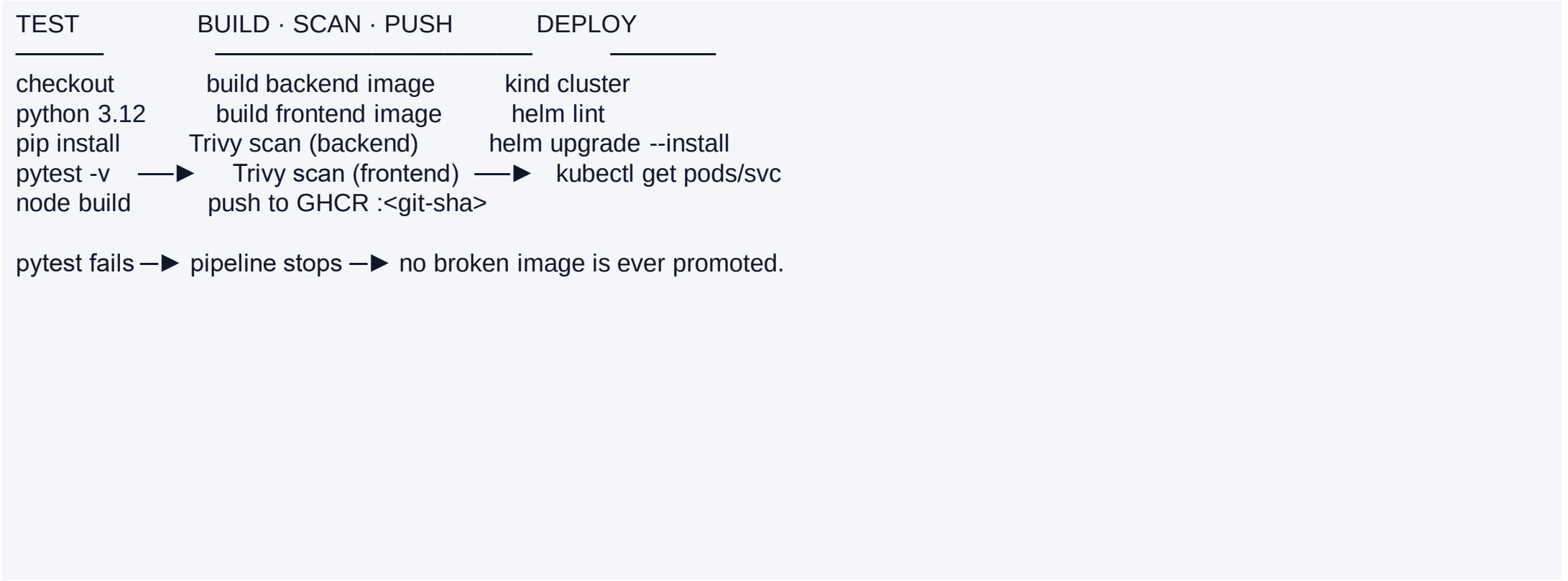
Application & Data Model

Backend surface

- Single primary table: appointments (patient, doctor, department, type, status, timestamps).
- 10 routes: GET / , /health , /ready , /metrics and full appointment CRUD + /stats.
- Status lifecycle: SCHEDULED → CHECKED_IN → COMPLETED (or CANCELLED).
- Pydantic validates enums and lengths → invalid input returns HTTP 422 automatically.
- Alembic migration 0001_create_appointments builds the table and a status index.

CI/CD Pipeline

GitHub Actions — three stages



Containers & Security Scanning

Build once, run as non-root

- Backend image — slim Python base, non-root UID 10001, waits for Postgres, runs migrations on start.
- Frontend image — multi-stage: node build → static dist → unprivileged Nginx (UID 101, port 8080).
- Both images run as non-root users and expose health endpoints for Kubernetes probes.
- Trivy scans both images for OS + dependency CVEs at HIGH/CRITICAL, exit-code 1 → the build fails on findings.

Infrastructure as Code

Terraform → AWS VPC + EKS

```
module "vpc"   →  VPC 10.20.0.0/16
                  ├── 2 public  subnets  (10.20.101/102.0/24)
                  ├── 2 private subnets  (10.20.1/2.0/24)
                  └── NAT gateway

module "eks"   →  EKS cluster (k8s 1.31)
                  └── managed node group (2 × t3.medium, 2–4 auto)

terraform init → plan → apply → destroy
```

Kubernetes & Helm

Packaged, versioned, autoscaled

- Helm chart helm/clinicflow — Chart.yaml, values (+ dev/prod overrides), templates.
- Backend & frontend Deployments run 2 replicas with readiness and liveness probes.
- ClusterIP Services for stable networking; Ingress routes / → frontend and /api → backend.
- Horizontal Pod Autoscaler scales the backend 2 → 6 on 60% CPU utilisation.
- ServiceMonitor wires the backend /metrics endpoint into Prometheus.

Observability

Prometheus + Grafana

- Backend exposes Prometheus metrics on /metrics (prometheus-fastapi-instrumentator).
- kube-prometheus-stack installed via Helm; ServiceMonitor tells it what to scrape.
- Grafana dashboard (auto-loaded) shows request rate, p95 latency and responses by status code.
- Alerts on traffic, errors and CPU; HPA reacts to the same signals under load.

Key Learnings

What this capstone demonstrated

- A pipeline is only trustworthy if it fails loudly — pytest and Trivy both gate the build.
- Tagging images with the Git commit SHA gives traceability from production back to source.
- Infrastructure as code makes environments reproducible instead of hand-clicked.
- Declarative Kubernetes + Helm turns deployment into a versioned, reviewable artefact.
- Observability closes the loop — you cannot operate what you cannot measure.
- Non-root containers are cheap to do and cut a whole class of risk.